

Anthropic 产品负责人：从 6 个月到 1 天的发版秘密，harness 会被模型当早餐吃掉

原创 John AGI Hunt 2026年4月25日 12:05 北京



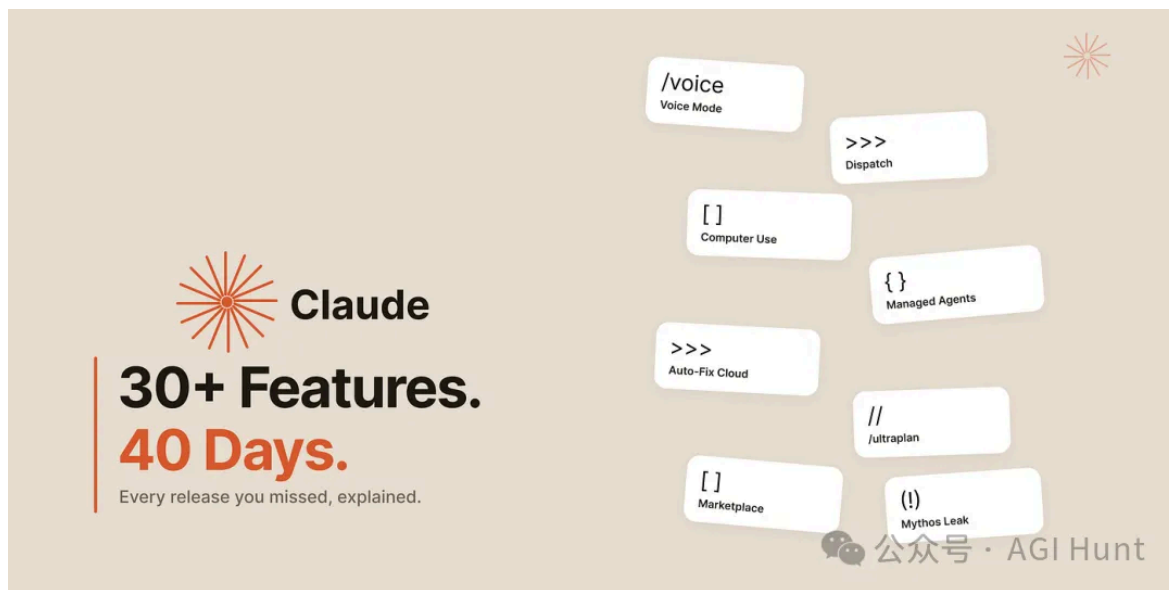
AGI Hunt

关注AGI 的沿途风景！ 前网易资深技术专家；出海某垂类 Agent 方向创业中；佛系写作
1815篇原创内容

公众号

可以说，Anthropic 的产品发布节奏，算得上是独一份的了。

如果你做上一张日历图，把 Anthropic 最近的产品发布标出来，那就会发现：几乎每天都有一个新功能上线。



Anthropic 40 天发布了 30+ 功能

最近，Lenny Rachitsky 请到了 Kat Wu，Anthropic Claude Code 和 Cowork 的产品负责人，访谈了一期播客。节目信息密度相当高，从 PM 角色的变化、Anthropic 的内部流程，到源码泄露事件和 OpenClaw 决策，全都聊了个遍。

我把里面的关键信息拎了出来，分享如下：



Cat Wu, Anthropic Claude Code 产品负责人

01

她是谁

Kat Wu 之前做了多年工程师，短暂做过 VC，后来加入 Anthropic，成为 Claude Code 和 Cowork 的产品负责人。

她和 Boris（Claude Code 的创造者和技术负责人）搭档。

Boris 负责产品愿景，定义三到六个月后产品应该长什么样。Kat 则负责把愿景拆解成可执行路径，并协调市场、销售、财务等各个团队。



“ 我们大概 80% 的想法是一致的。剩下的 20%，谁更在意就谁来推。

她们团队有一个特点：几乎所有 PM 都有工程背景，设计师也曾是前端工程师。

这倒不是刻意为之。Kat 的解释是，有工程背景的人能更快判断一个东西做起来到底有多难，而这个判断在当前的节奏下实在是太关键了。

02

快到什么程度

Anthropic 的产品迭代周期，从六个月压到了一个月，有些功能甚至只用一天。

而这背后的秘诀，倒也没有复杂的方法论。

Kat 提到了三件事：



公众号 · AGI Hunt

Anthropic 发版三板斧流程图

设定清晰的目标。

LLM 太通用了，如果不锁定用户和场景，团队很容易迷失方向。比如 Claude Code 的目标用户是专业开发者，而某个功能要解决的问题是「权限弹窗太多导致疲劳」，目标是让企业开发者能安全地实现零权限提示。

这一条就排除了大量不相关的方案。

Research Preview 机制。

几乎所有功能都以 research preview 的形式先上线。用户知道这是一个早期版本，可能不会永久保留。这样做的好处是，团队不需要做到完美才能发布，一两周就能把东西推出去。

Launch Room 流程。

工程师觉得功能差不多了，就把它丢进一个叫「evergreen launch room」的频道。Docs 负责人 Sarah、PMM 负责人 Alex、DevRel 团队的 Tarek 和 Lydia 会迅速跟进，第二天就能发布。

“ 我们希望移除一切阻碍发布的障碍。团队里每个人都应该能在一周之内，甚至一天之内，把自己的想法变成产品。

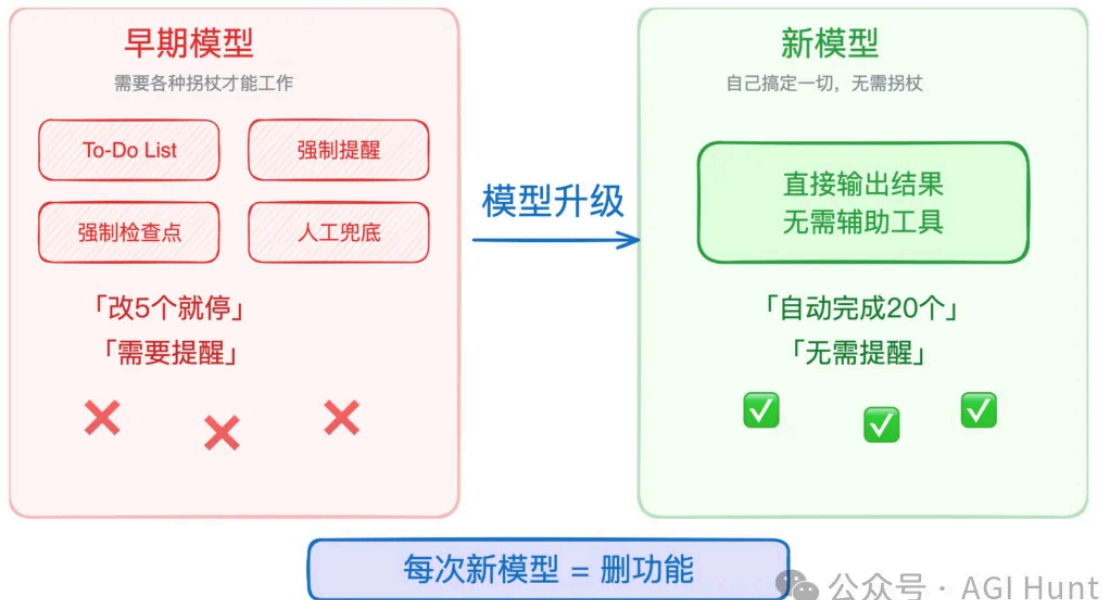
Lenny 忍不住追问：你们内部用了 Mythos 模型……是不是因为这个才快的？

Kat 的回答是：

“ 我们确实在内部用了这些模型，它确实加快了一点速度。但大部分的加速来自流程和团队文化。

03

模型会吃掉你的产品



早期模型需要拐杖 vs 新模型自己搞定

Kat 聊到了一个值得注意的现象：每次新模型出来，她们做的第一件事，是删功能。

Claude Code 最早有个 to-do list 功能。当时的模型在做大规模代码重构时，总是改了 5 个调用点就停了，明明有 20 个要改。团队想了个办法：让模型先列个清单，然后逐个完成。

结果这招非常管用。

但到了 Opus 4 之后呢，模型自己就会主动列清单、逐个完成了。to-do list 从一个「必要的拐杖」变成了「可有可无的辅助界面」。

“ 每次发布新模型，我们都会通读整个 system prompt，逐段反思：模型还需要这个提醒吗？如果不需要了，就删掉。

Lenny 举了个例子：「模型会把你的 harness 当早餐吃掉。」

Kat 表示同意。

但更让她兴奋的是新模型解锁的全新能力。比如代码审查：她们试了好几个版本，准确率一直不够。直到 Opus 4.5 和 4.6 出来，才达到了让工程团队真正依赖它的水平，现在 Anthropic 内部合并 PR 之前，必须先过 Claude 的代码审查。

“提前做好还没完全能用的产品原型很重要。这样新模型一出来，你直接换上去看看差距有没有被弥合，就可以了。

这和之前 Mike Krieger 在播客里说的是一样的思路：**为未来的模型做产品。**

04

源码泄露

上个月 Claude Code 源码泄露的事情，Kat 也做了回应。

“我们第一时间就排查了。这是人为失误的结果，有人在用 Claude 写 PR 更新包发布流程时出了差错。虽然经过了两层人工审查，但还是漏了。

Lenny 问那个人还在吗，Kat 的回答很是体现了 Anthropic 的文化：

“这是流程问题。最重要的是从中学习，加上更多防护措施。大部分改进已经上线了。

05

OpenClaw 的抉择

Lenny 还问道了 OpenClaw。最近 Anthropic 限制了第三方工具（如 OpenClaw 🦜️）使用 Claude 订阅额度，社区一片哗然、抗议。

Kat 的解释是：Claude 的需求增长太快，订阅额度本来是为第一方产品设计的。第三方工具的使用模式不同，给基础设施带来了额外的压力。

“我们花了很多时间研究怎么做最平滑的过渡。能给每个用户一些 credits 作为缓冲，这让我比较欣慰。但我们确实需要优先保障第一方产品和 API。

Lenny 自然是站在 Anthropic 这边：

“ 你们在 200 美元月费下提供几乎无限的使用量，本身就在补贴用户。公司也需要盈利。

06

Cowork 被低估了

节目里还有一段，是关于 Cowork 的。

Kat 用 Cowork 给即将到来的 Code with Claude 大会做了一份 20 页的演讲稿。她的做法是：连接好 Google Calendar、Slack、Gmail 和 Google Drive，然后告诉 Cowork 演讲的主题和叙事方向。

Cowork 花了大约一个小时，翻阅了 X 上的产品发布记录、团队内部的 launch room 频道和 demo 频道，自己整理出一份 20 页的演示文稿。

“ 我早上起来读了一遍，还挺好的。文字稍微多了点，做了一轮反馈调整。但视觉上看起来就像 Anthropic 的设计师做的一样，因为 Cowork 能读取我们的设计系统模板。

她把产品分成两类来用：**输出是代码的，用 Claude Code。输出不是代码的（PPT、文档、邮件），用 Cowork。**

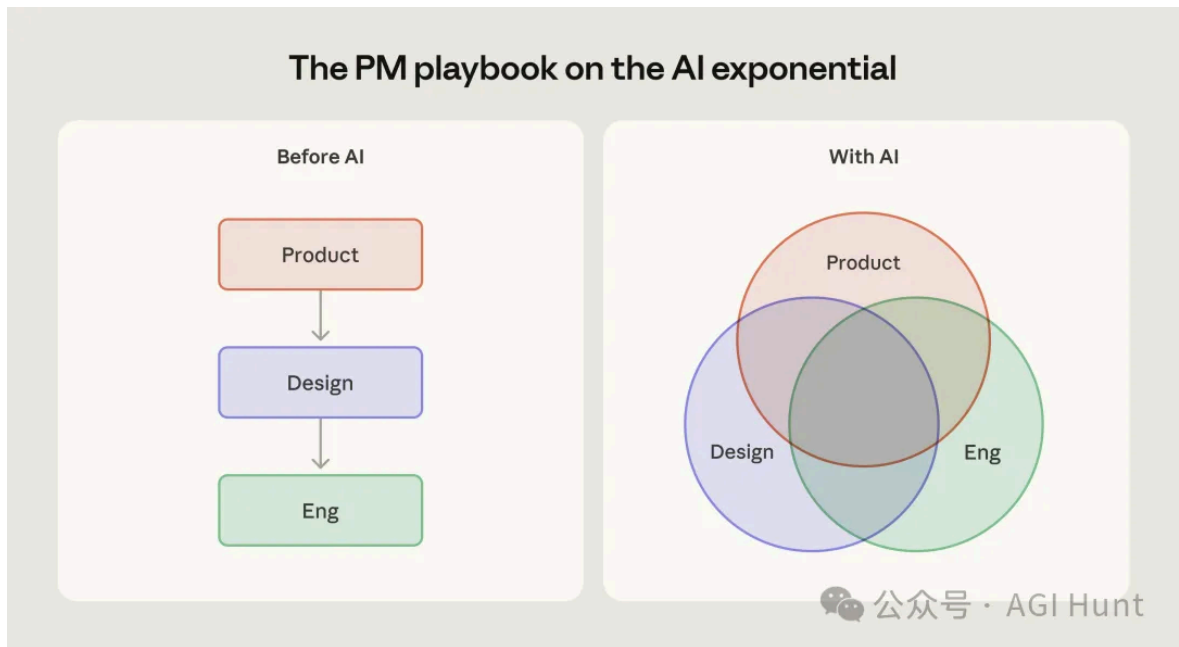
Applied AI 团队（负责帮客户采用 Claude API 的技术型市场团队）应该是 Anthropic 内部除工程师之外最大的 token 消耗者。

他们用 Cowork 在每次客户会议前自动生成 briefing：明天要见哪些客户、他们之前问过什么、Action items 是什么、某个功能的最新上线时间是什么。

这些都是他们自己搭建的工作流，做好了之后分享给团队其他人。

07

PM 的未来



AI 时代 PM 角色融合示意图

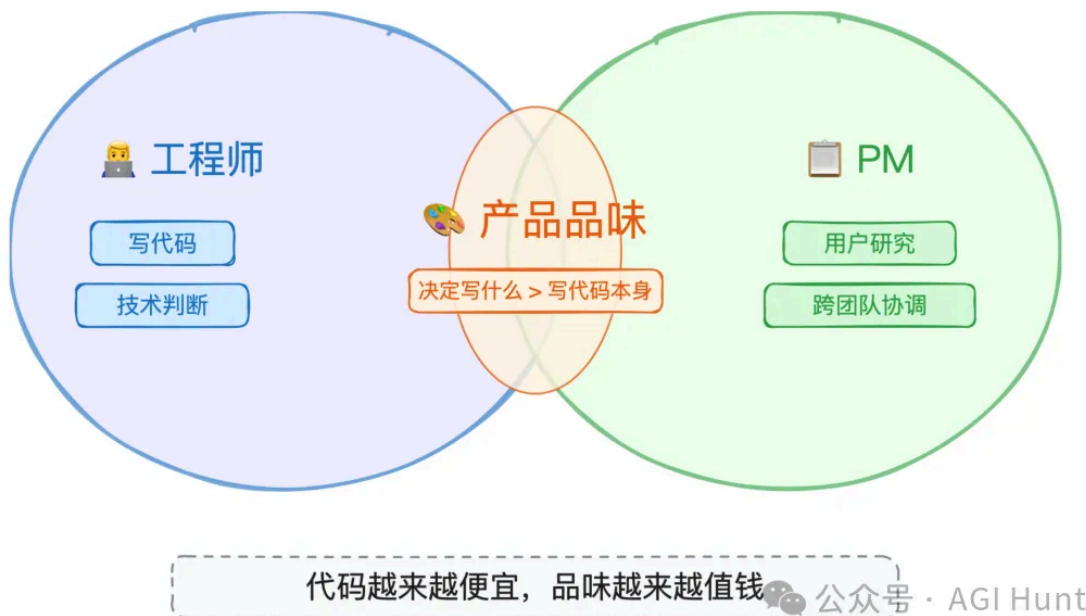
Kat 对 PM 角色变化的看法，是这期节目里最应该记下来的部分。

“ 代码变得越来越便宜了。那什么变得更有价值呢？决定写什么。

她说 Anthropic 目前更倾向于招有产品品味的工程师，传统意义上的 PM 反而不是第一选择。

团队里有不少工程师可以从看到 X 上的用户反馈开始，到周末就自己上线一个功能，几乎不需要 PM 参与。

“ PM 和工程师这两个角色在融合。你多招些有产品品味的工程师，或者多招些 PM 来指导工程方向，效果差不多。



PM 与工程师的产品品味交集

但融合的代价呢？

产品一致性。

有时候同一个需求会有两个功能在做，因为团队内部有两种方案都觉得好，干脆都上线让用户来投票。对新用户来说，这意味着要花更多时间弄清楚该用什么。

Kat 坦言，/powerup 功能（一个内置的教程引导）其实违背了他们最初「产品应该直觉到不需要教程」的原则。但功能实在太多了，用户确实需要有人告诉他们：这一百个功能里，哪十个是必须用的。

08

问模型为什么犯错

Kat 分享了一个做 AI 产品的独特技巧：**让模型自己反思它的行为。**

比如她发现模型改了前端代码后会跑测试，但.....就是不会真的打开页面看一眼 UI。她就问模型：你为什么不检查 UI 呢？

模型的回答有时候会让人意外：

“ 有时候模型会说，system prompt 里的某段话让它产生了困惑。有时候它会说，我把验证任务委派给了 sub-agent，但 sub-agent 没做，而我也没有检查它的工作。

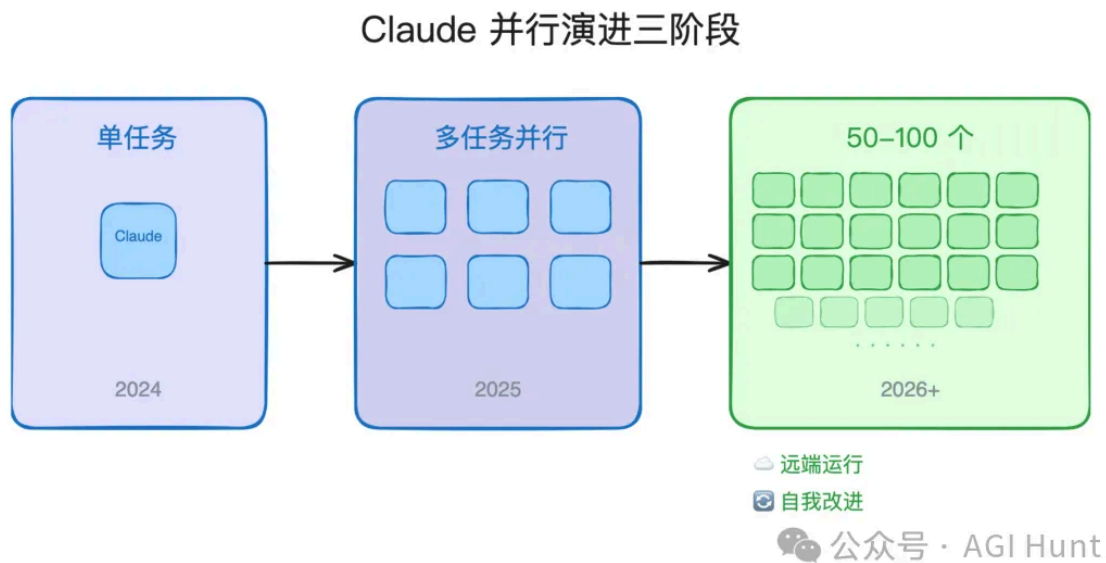
这些反馈，则直接指向了 harness 层面的改进方向。

“ 对模型的决策保持好奇心，问它为什么做出了那个选择，你就能看到是什么误导了它，然后修复 harness 来填补这个空隙。

09

50 个 Claude 并行

聊到未来的路线图，Kat 把产品演进拆成了几个阶段，像搭积木一样：



Claude 并行演进三阶段

第一步：单个任务成功。给一个清晰的 prompt，模型能不能稳定地输出可以合并的代码或者可以分享的文档？

第二步：多任务并行。2025 年底 multi-coding 就已经开始火了。目前用户大概同时跑 6 个 Claude。

第三步：50 到几百个 Claude 同时跑。到那时候，本地机器的内存肯定是不够用了，任务得跑在远端。界面需要告诉你哪些任务需要你看一眼，而且模型应该能自己验证工作，这样你看到「已完成」的时候可以放心地信任它。

“而且这个过程要能自我改进。你给了一次反馈，模型在之后的每次运行中都不会再犯同样的错误。

10

Just do things

Kat 的人生座右铭是：**Just do things**。

“工作本来就是假的。如果你理解了约束条件，你就能想出该做什么，然后就去做。快速行动，从错误中学习，如果做错了就道歉并修复。

这话放在 Anthropic 的语境下不难理解：她说很多公司里角色界限分明，PM 做 PM 的事，工程师做工程师的事。

而 Anthropic 鼓励的是跨界：**你看到一个问题，不用管它属于谁的地盘，直接解决它。**

11

别停在 95%

最后一个值得记下来的建议，是关于自动化的。

Kat 说她见过两种极端：一种人从来不做自动化，另一种人沉迷于折腾工具配置、加 MCP、搞 Skills，花的时间比真正完成任务还多。

而更常见的问题是……很多人把自动化做到 95% 之后，就放弃了。

“如果一个自动化不能 100% 工作，那它就不是真正的自动化。最后那 5% 确实需要更多时间，但你得投入这个精力，教 Claude 你的偏好，给它反馈，直到它能完全可靠地运行。

她自己也承认在用 Cowork 做 Gmail 收件箱清零时，也还没做到 100%。

但这就是方向：找到你重复做的、不喜欢做的事情，交给 AI，然后把它打磨到完全可靠。省下来的时间，去做你真正想做的事。

这才是 AI 给每个人的真正杠杆。



相关链接：

- Lenny's Podcast 原文：<https://www.lennysnewsletter.com/p/how-anthropics-product-team-moves>
- YouTube 完整视频：<https://www.youtube.com/watch?v=PplmzlgE0kg>
- Kat Wu X：https://x.com/_catwu

 喜欢作者